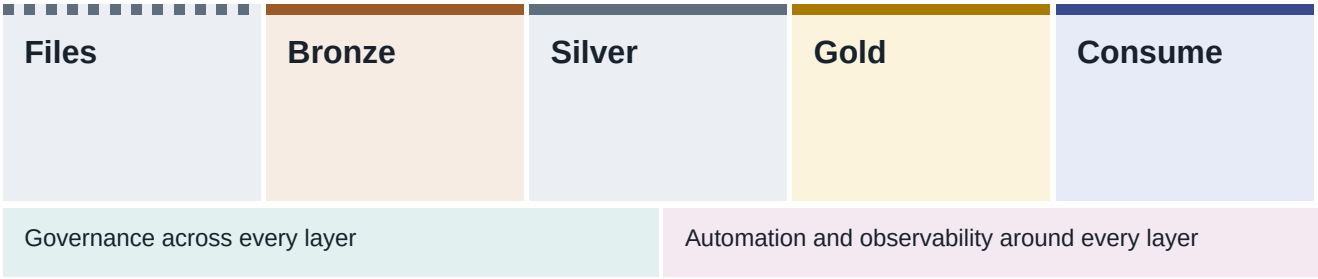
Data Engineering capstone project

# Project Meridian

## Build a governed lakehouse, end to end, on Databricks Free Edition




| | |
|---|---|
| Audience | **Trainees, individual work** |
| Platform | **Databricks Free Edition, one workspace per trainee** |
| Duration | **10 working days (trainer may adjust)** |
| Practice | **Data Engineering, training content** |
| Your dataset | **Assigned separately by your trainer** |

This brief tells you what to build and how you will be assessed. It deliberately does not tell you how: researching the Databricks documentation and making justified design decisions is part of the assignment.

# 1. Problem statement

Meridian Holdings is a fictional group of business units: taxi operations, wholesale trading, retail stores, a bakery franchise, a holiday-rental platform, a weather intelligence service and more. Each unit has data. None of them trusts it.

### The situation

Meridian grew by acquisition. Every business unit exports its operational data as files on its own schedule and builds its own reports on private copies of those files. The consequences are familiar:

- The same KPI has several definitions, so the board receives conflicting numbers.
- Analysts re-clean the same columns every week, each in a slightly different way.
- Nobody can trace a reported number back to the file it came from.
- Sensitive fields such as phone numbers, email addresses and card numbers sit unmasked in spreadsheets.
- Regional managers can see every region's data because there is no practical way to restrict it.
- When a nightly refresh fails, the business finds out from an angry email rather than from monitoring.

### The mandate

The Group Chief Data Officer has commissioned a pilot. Each business unit is given one data engineer (you) and one Databricks workspace. In ten working days you must prove that a single lakehouse pattern, the Medallion Architecture, fixes every problem above for your unit. Your trainer assigns your business unit and its source dataset separately.

The CDO will judge the pilot on five outcomes:

| Problem today | What you must deliver |
|---|---|
| Raw files are overwritten and history is lost | An append-only, replayable raw layer that records which file every row came from |
| Every analyst cleans data differently | One validated, deduplicated, historised layer, plus a no-code preparation path for analysts |
| KPIs have several definitions | One governed definition of each KPI, reused by every reporting tool |
| Sensitive data is exposed; access is all-or-nothing | Classified data, rows restricted by business dimension, masked sensitive columns |
| Refreshes are manual; failures go unnoticed | Code-deployed, scheduled, monitored pipelines with self-service consumption |

# 2. Objectives

### Business objectives

1. Deliver a trustworthy, single source of truth for your business unit's KPIs.
2. Make data available to business users through dashboards, natural-language questions and Power BI, without exposing data they are not entitled to see.
3. Run the whole flow automatically, detect bad data before it reaches users, and show operational health at a glance.

### Learning objectives

By the end of the project you will be able to:

1. Design and justify a Bronze, Silver and Gold Medallion Architecture for a real dataset.
2. Build incremental, declarative pipelines with data quality rules and change data capture.

3. Model a Gold layer and define reusable business metrics in Unity Catalog.
4. Govern data with Unity Catalog: classification, row-level and column-level security, least-privilege access.
5. Deploy and orchestrate a data product as code, with parameters, dependencies and quality gates.
6. Serve data through SQL, AI/BI dashboards, a Genie Agent and an external BI tool.
7. Monitor a data platform using Databricks system tables.
8. Find, read and apply official documentation for features that change frequently.

# 3. Mandatory features

Your solution must use every feature below. Several products were renamed in 2025 and 2026; search for both names when researching, and use the current name in your deliverables. A missing mandatory feature caps your total score at 60 out of 100.

| Area | Feature (current name, former name) | What it must achieve in your solution |
|---|---|---|
| Orchestration | Declarative Automation Bundles (formerly Databricks Asset Bundles, still "DABs") | Your whole project deploys from Git with one command |
| | Lakeflow Jobs | One scheduled, parameterised job runs the end-to-end flow |
| Processing | Lakeflow Spark Declarative Pipelines (formerly Delta Live Tables) | Bronze, Silver and Gold built declaratively with quality rules |
| | Visual data prep in Lakeflow Designer | An analyst-style transformation built on the no-code canvas |
| Storage | Delta Lake tables | All layers stored as Delta; history and time travel demonstrated |
| | Unity Catalog Managed Volumes | Landing zone for incoming files |
| Governance | Governed tags | A controlled classification vocabulary applied to your data |
| | Row-level security (row filters) | Users see only rows for the business dimension they are entitled to |
| | Column-level security (column masks) | At least two sensitive columns masked for unauthorised users |
| BI and analytics | Metric views | Each KPI defined once and reused everywhere |
| | SQL Editor with parameters | Reusable, typed, parameterised analysis queries |
| | AI/BI dashboards | A Business KPI dashboard for business users |
| Observability and AI | System tables | An operational dashboard on platform telemetry |
| | Genie Agents (formerly Genie spaces) | Natural-language questions answered over your Gold layer |
| Integration | Power BI | An external report reading your governed Gold data |

# 4. Platform constraints

You will build on Databricks Free Edition, which offers most of the platform on serverless compute with usage quotas. Read the official *Databricks Free Edition limitations* page before designing anything. The constraints that most affect this project are:

- **Serverless only.** No cluster configuration; Python and SQL only.
- **One active pipeline per pipeline type.** Design your Medallion layers so this limit does not block you.
- **At most five concurrent job tasks** per account.
- **One SQL warehouse, 2X-Small.** Dashboards, Genie, SQL tasks and Power BI all share it.
- **Daily usage quota.** When exceeded, compute stops for the rest of the day; data is kept. Work with small data volumes.
- **No DBFS root access.** Do not use `/databricks-datasets` or `dbfs:/` paths. Your source data is in the read-only `samples` catalog.
- **You are the only user** and the workspace admin. You will test security by changing your own group memberships.

> **Research rule**
>
> Features in this brief evolve quickly. Base your work on the official Databricks documentation current at the time you build, not on older blog posts or courses. When the documentation and this brief disagree, follow the documentation and record the difference in your README.

# 5. Initial setup: catalog, schemas and volume

This is the only part of the project where commands are provided. Everything after it is yours to research and build. Replace `NN` with your two-digit trainee number (for example `07`) everywhere.

### Naming conventions

| Object | Name |
|---|---|
| Catalog | meridian_tNN |
| Schemas | landing, bronze, silver, gold, gov (security objects), ops (quality and monitoring) |
| Managed Volume | landing.raw_files (path /Volumes/meridian_tNN/landing/raw_files/) |
| Groups | tNN_admins (see all rows), tNN_pii_readers (see unmasked sensitive columns) |
| Pipeline, job, bundle | meridian_tNN_pipeline, meridian_tNN_job, meridian_tNN |
| Gold objects | dim_<name>, fct_<name>, vdp_<name> (visual data prep output), mv_<name> (metric views) |

### Run in the SQL Editor

SQL Editor: initial setup

```
-- 1. Confirm identity and the sample data you will use
SELECT current_user() AS me, current_version() AS dbsql_version;
SHOW SCHEMAS IN samples;

-- 2. Catalog for your project
CREATE CATALOG IF NOT EXISTS meridian_tNN
  COMMENT 'Project Meridian capstone, trainee NN';
USE CATALOG meridian_tNN;

-- 3. One schema per layer, plus security and operations
CREATE SCHEMA IF NOT EXISTS landing COMMENT 'Raw file drops';
CREATE SCHEMA IF NOT EXISTS bronze  COMMENT 'Append-only raw tables';
CREATE SCHEMA IF NOT EXISTS silver  COMMENT 'Validated, conformed tables';
CREATE SCHEMA IF NOT EXISTS gold    COMMENT 'Business-ready models and metric views';
CREATE SCHEMA IF NOT EXISTS gov     COMMENT 'Security functions and entitlement tables';
CREATE SCHEMA IF NOT EXISTS ops     COMMENT 'Data quality results and monitoring';

-- 4. Managed Volume for incoming files
CREATE VOLUME IF NOT EXISTS landing.raw_files
  COMMENT 'Landing zone for batch file drops';

-- 5. Check
SHOW SCHEMAS IN meridian_tNN;
SHOW VOLUMES IN meridian_tNN.landing;
```

Then create the two groups from **Settings > Identity and access > Groups** (workspace admin settings) and add yourself to `tNN_admins` only.

> **If catalog creation is not allowed**
>
> If `CREATE CATALOG` fails in your workspace, use the built-in `workspace` catalog and prefix each schema with `tNN_` (for example `workspace.t07_bronze`). Tell your trainer and record the deviation in your README.

**Later in the project**

In Phase 5 your bundle must own or reference these schemas and the volume. Research how bundles treat objects that already exist, and choose an approach: recreate them from the bundle, bind the existing ones, or reference them as variables. Justify your choice.

# 6. Step-by-step instructions

Work through the phases in order. Each phase states an objective, the steps to complete, topics to research, and the evidence you must collect. The steps say *what* to achieve; working out *how* is part of the assignment.

**PHASE 0** Day 1

## Explore and plan

**Objective.** Understand your assigned dataset and the platform before writing any pipeline code.

**Steps**

1. Complete the initial setup in Section 5.
2. Tour the workspace: Catalog Explorer, SQL Editor, Jobs & Pipelines, Dashboards, Genie and the Previews page. Note any features that are missing or in preview.
3. Profile your assigned source tables: row counts, keys, date ranges, null rates, candidate dimensions and sensitive columns. Confirm every column name with `DESCRIBE`.
4. Write a short design note: your entities, which are facts and which are dimensions, the grain of each, your row-filter dimension, your sensitive columns, and your KPIs.
5. Create a Git repository for the project and connect it to your workspace. Install and authenticate the Databricks CLI on your machine.
6. Check which system tables you can read (Phase 7 depends on them) and record the result.

**Research**

Free Edition limitations; Unity Catalog object model (catalog, schema, table, volume); the samples datasets; Git folders; Databricks CLI installation and OAuth authentication; system tables overview.

**Evidence for review**

- Data profile and design note in your README.
- Screenshot of your catalog, schemas and volume.
- List of system tables readable in your workspace.

PHASE 1 Days 1 to 2

## Landing and Bronze

**Objective.** Simulate realistic file arrivals and ingest them incrementally into an append-only Bronze layer.

**Steps**

1. Build a landing simulator notebook that reads your sample tables and writes them into the Managed Volume as files, one folder per entity and one sub-folder per numbered batch.
2. Split fact data into disjoint batches so each run delivers new records. Land dimension data once.
3. Make the simulator inject a small amount of bad data in each batch: some null keys, some duplicates, some invalid values. Silver must catch them later.
4. Make the simulator parameter-driven (catalog, batch number, rows per batch) so a job can call it. Keep batches small: a few thousand rows.
5. Create one Lakeflow Spark Declarative Pipeline for your project. Define a Bronze streaming table per entity that ingests new files incrementally.
6. Add lineage columns to every Bronze table: source file path, ingestion time and batch number. Do not filter, deduplicate or reshape business columns in Bronze.
7. Land at least three batches and prove each pipeline update processes only the new files.
8. Demonstrate Delta Lake table history and query an earlier version of a Bronze table.

**Research**

Writing files to Unity Catalog volumes; Auto Loader and the read_files function; streaming tables in pipelines; the file metadata column; pipeline configuration parameters; publishing to multiple schemas from one pipeline; DESCRIBE HISTORY and time travel.

**Evidence for review**

- Volume folder structure with three batches.
- Pipeline graph and row counts per update.
- History and time-travel query results.

PHASE 2 Days 3 to 4

## Silver and Visual data prep

**Objective.** Turn raw data into trustworthy data, and give analysts a governed no-code path on top of it.

**Steps**

1. Write a Silver contract in your README: naming, data types, validity rules, uniqueness and history rules.
2. Rename columns to business-friendly names and cast every column to its correct type.
3. Define at least three data quality expectations per fact table. Choose and justify the action for each (keep and record, drop the row, or fail the update).
4. Keep the rows you drop: capture them in a quarantine table in the `ops` schema, and show the counts reconcile with Bronze.
5. Guarantee one row per business key in Silver.
6. Historise at least one dimension as Slowly Changing Dimension Type 2 using the pipeline's change data capture feature. Land a batch that changes some dimension attributes to prove it works.
7. Build a Visual data prep in Lakeflow Designer that answers your assigned analyst question from Silver data. Use at least four different operator types and one parameter, and write the result to `gold.vdp_<name>`.
8. Inspect the code Lakeflow Designer generated, create a job for the visual data prep, and version the prep file in your repository.

**Research**

Pipeline expectations and violation actions; quarantine patterns; deduplication in streaming and materialised datasets; AUTO CDC (formerly APPLY CHANGES) and SCD Type 2 output columns; sequencing change records; Lakeflow Designer and visual data prep (Public Preview); moving a visual data prep to production.

**Evidence for review**

- Expectation metrics from the pipeline UI or event log.
- Reconciliation: Bronze rows versus Silver plus quarantine.
- SCD Type 2 query showing versioned keys.
- Visual data prep canvas, generated code excerpt and output table.

PHASE 3 Days 4 to 5

## Gold and metric views

**Objective.** Model the business and define every KPI exactly once.

**Steps**

1. Design a small star schema: at least one fact table with a declared grain and two conformed dimensions. Include your row-filter dimension on the fact.
2. Build the Gold tables in your pipeline as materialised datasets, with a comment on every table stating its grain and on key columns stating their meaning.
3. Run the whole pipeline from Bronze to Gold in a single update.
4. Create a Unity Catalog metric view over your Gold fact with four to six measures from your assigned KPIs, including at least one ratio, and the dimensions users will slice by.
5. Describe every measure and dimension in Catalog Explorer.
6. Prove the metric view is correct: compute one KPI from the metric view and independently from the fact table, and show they match.

**Research**

Dimensional modelling (grain, facts, conformed dimensions); materialized views in pipelines; Unity Catalog metric views (YAML specification, MEASURE function, supported versions); why ratios must be defined as ratios of sums.

**Evidence for review**

- Star schema diagram.
- Pipeline graph covering all three layers.
- Metric view definition and reconciliation query result.

PHASE 4 Day 6

## Governance: tags, row filters, column masks

**Objective.** Classify your data and enforce who can see which rows and columns, everywhere the data is consumed.

**Steps**

1. Create a governed tag policy with at least three tags, for example sensitivity level, type of personal data, and Medallion layer, each with a fixed list of allowed values.
2. Apply the tags to your schemas and to every sensitive column.
3. Document your security model: who sees all rows, who sees which subset, and who sees sensitive columns unmasked.
4. Implement row-level security on your Gold fact and dimension using a row filter. Store entitlements in a mapping table in the `gov` schema rather than hard-coding them.
5. Implement column-level security on at least two sensitive columns using column masks, with different masking rules where appropriate (partial masking, full redaction).

6. Attach the controls to your pipeline-managed Gold tables. Research how this differs from ordinary Delta tables.
7. Check that a table built from a row-filtered table is not itself silently filtered. Think about whose identity the pipeline uses during refresh.
8. Test every control by changing your own group memberships and entitlements, re-running the same query each time.
9. Grant consumers access to Gold only. Write an audit query proving every column tagged as sensitive has a mask.
10. Stretch goal: replace explicit masks with a tag-driven attribute-based access control policy, if available.

**Research**

Governed tags and tag policies; SET TAG; row filters and column masks (SQL UDFs, mapping tables, is_account_group_member, current_user); row filters and masks on pipeline tables; Unity Catalog privileges; information_schema views for tags and masks; attribute-based access control.

**Evidence for review**

- Governed tag definitions and the audit query result.
- Before and after screenshots for the row filter and each mask.
- Security model table in your README.

PHASE 5 Day 7

## Orchestration with bundles and Jobs

**Objective.** Replace every manual step with code: one bundle deploys the project, one job runs it.

**Steps**

1. Create a Declarative Automation Bundle with variables (catalog, warehouse, trainee) and at least two targets: a development target whose schedules deploy paused, and a production target.
2. Define your schemas, volume, pipeline, job and dashboards as bundle resources. Decide how to handle the objects created in Section 5.
3. Design a job that runs, in order: governance setup, file landing, the pipeline, metric views and the visual data prep, then a data quality gate. Respect the five-concurrent-task limit.
4. Use job parameters so a run can land a chosen batch or automatically pick the next one.
5. Write a data quality gate task that records results in an `ops` table and fails the run when a check fails (for example: empty Gold, duplicate keys, orphan fact rows).
6. Add a schedule and a failure notification.
7. Deploy from the CLI. Run the job successfully, then break a quality check on purpose, observe the failure, fix it and repair only the failed task.

**Research**

Declarative Automation Bundles: structure, variables, targets, presets, resources for schemas, volumes, pipelines, jobs and dashboards; bundle validate, deploy, run and generate; job task types (notebook, pipeline, SQL file, run job); job parameters and dynamic values; repair runs; serverless jobs on Free Edition.

**Evidence for review**

- Repository with the bundle.
- Output of bundle validate and deploy.
- Job DAG, one successful run, one failed run and one repaired run.

PHASE 6 Day 8

## SQL Editor and Business KPI dashboard

**Objective.** Serve governed KPIs to business users.

**Steps**

1. Write and save three parameterised queries in the SQL Editor, with typed parameters (text, date, number). At least one must query the metric view.
2. Build an AI/BI dashboard named for your business unit, using the metric view as its main dataset.
3. Include at least one parameter, one filter, four KPI counters, a monthly trend, a breakdown by your row-filter dimension, a detail table and a text box defining the KPIs.
4. Use the dashboard's AI assistance to draft one chart, then review and correct it.
5. Decide how to publish so that row filters and masks still apply to each viewer, and justify the decision.
6. Schedule a refresh, then demonstrate the dashboard under restricted access.

**Research**

Named parameter markers in the SQL Editor; AI/BI dashboards (datasets, parameters versus filters, metric views as datasets); publishing options and embedded credentials; dashboard schedules.

**Evidence for review**

- Three saved queries.
- Published dashboard.
- Screenshot of the dashboard under restricted access.

PHASE 7 Day 8

## Operations dashboard on system tables

**Objective.** Show the CDO that the platform is healthy, and where it is not.

**Steps**

1. Identify the system tables that answer these questions: Did the last run succeed? Which task is slowest? How long do pipeline updates take? What consumes compute? Which queries are slow? Is data quality getting worse? Where does Gold data come from?
2. Inspect each table's columns before querying it.
3. Use the pipeline event log to report expectation pass and fail counts per update.
4. Build a two-page operations dashboard: run health, and cost and usage. Include your `ops` quality results.
5. Make sure the deliberate failure from Phase 5 is visible.
6. If a system schema is not readable on Free Edition, design and document an alternative source.
7. Stretch goal: an alert that fires when the latest run fails.

**Research**

System tables reference (billing, lakeflow jobs and pipelines, query history, lineage); the pipeline event log; SQL alerts.

**Evidence for review**

- Operations dashboard with at least six widgets.
- Note of any system tables you could not use and your alternative.

PHASE 8 Day 9

## Genie Agent

**Objective.** Let business users ask questions of their data in plain English, safely.

**Steps**

1. Create a Genie Agent over your metric view and two to four Gold tables. Do not add Bronze or Silver.
2. Write instructions in business language: what the unit does, KPI meanings, code values, default time window, and what to do when a question is out of scope.
3. Add at least three example SQL queries for common questions.
4. Create at least five benchmark questions with expected answers; reach at least four out of five correct.
5. Test: a KPI question, a follow-up, an ambiguous question, an out-of-scope question, a request for sensitive data, and a question while your access is restricted.

**Research**

Genie Agents (formerly Genie spaces): data selection, instructions, example SQL, benchmarks, how Genie applies the user's permissions.

**Evidence for review**

- Agent configuration screenshots.
- Benchmark score.
- Results of the six tests.

PHASE 9 Day 9

## Power BI integration

**Objective.** Prove the lakehouse serves external tools with the same numbers and the same security.

**Steps**

1. Connect Power BI Desktop (Windows) to your SQL warehouse using the Databricks connector. Find the connection details and choose an authentication method.
2. Choose Import or DirectQuery and justify the choice for a 2X-Small warehouse.
3. Use the metric view for your KPIs, or rebuild them in DAX and prove they reconcile.
4. Build one report page: KPI cards, a monthly trend, a breakdown by your row-filter dimension and a date slicer.
5. Show one KPI matches your AI/BI dashboard, and that a refresh under restricted access returns only permitted rows and masked columns.
6. Never store credentials in the report file or your repository. Revoke any token you created when you finish.

**Research**

Connecting Power BI Desktop to Databricks; Partner Connect; personal access tokens versus OAuth; querying metric views from Power BI; Import versus DirectQuery.

**Evidence for review**

- Report file saved without credentials.
- Reconciliation and restricted-refresh screenshots.

PHASE 10 Day 10

# Production run and review

**Objective.** Show the whole data product working unattended, and present it.

### Steps

1. Deploy to your production target so the schedule is active, let one scheduled run complete, then pause the schedule.
2. Finish your README: design decisions, Silver contract, KPI definitions, security model, deviations from this brief and why.
3. Prepare a 20-minute demonstration: run the job, open both dashboards, ask the Genie Agent two questions, refresh Power BI, and change a group membership to show security.

### Research

Bundle targets and deployment; documenting architecture decisions.

### Evidence for review

- Scheduled run in the job history.
- Final README and evidence pack.

# 7. Deliverables

| Deliverable | Contents |
|---|---|
| Git repository | Your bundle and all source files. A clean bundle validate. No credentials anywhere, including history. |
| README | Data profile, design note, Silver contract, KPI definitions, security model, design decisions, deviations, and the official documentation pages you relied on. |
| Evidence pack | Screenshots for every phase, named by phase (for example p4_row_filter_after.png), and the Power BI file saved without credentials. |
| Live demonstration | 20 minutes, following the Phase 10 script. |

# 8. Evaluation

Total 100 points.

| Area | Points | Full marks requires |
|---|---|---|
| Landing and Bronze | 10 | Incremental ingestion proven over three batches; lineage columns; Delta history |
| Silver | 15 | Contract met; quarantine reconciles; working SCD Type 2 |
| Visual data prep | 5 | Four operator types, a parameter, output in Gold, run by the job |
| Gold and metric views | 15 | Declared grains; documented metric view; reconciliation |
| Governance | 15 | Governed tags applied; row filter and two masks enforced and demonstrated; clean audit |
| Bundles and Jobs | 15 | Deployable from Git; parameterised job; quality gate; failure and repair shown |
| SQL Editor and KPI dashboard | 10 | Typed parameters; dashboard on metric view; correct publishing choice |
| Operations dashboard | 5 | Six widgets from system tables or a documented alternative |
| Genie Agent | 5 | Instructions, examples, at least four of five benchmarks correct |
| Power BI | 5 | Reconciles with the dashboard; security shown on refresh |

- A missing mandatory feature caps the total at 60.
- Any credential found in the repository, README, screenshots or report file means resubmission.
- Justified deviations from this brief, recorded in the README, are not penalised.

# 9. Suggested schedule

| Day | Focus | Phases |
|---|---|---|
| 1 | Setup, workspace tour, data profiling, design note, landing simulator | 0, 1 |
| 2 | Bronze ingestion, incremental runs, Delta history | 1 |
| 3 | Silver: types, expectations, quarantine, deduplication | 2 |
| 4 | SCD Type 2, Visual data prep, start Gold | 2, 3 |
| 5 | Gold star schema and metric views | 3 |
| 6 | Governed tags, row filters, column masks | 4 |
| 7 | Bundle and job, quality gate, failure and repair | 5 |
| 8 | SQL Editor, KPI dashboard, operations dashboard | 6, 7 |
| 9 | Genie Agent and Power BI | 8, 9 |
| 10 | Production run, README, rehearsal, review | 10 |

# 10. Rules

- Work individually in your own Free Edition workspace. Discuss ideas with colleagues, but build your own solution.
- Use only your assigned dataset from the `samples` catalog.
- Never share or commit credentials, tokens or passwords. Use placeholders such as `REPLACE_WITH_TOKEN` in anything you share.
- Protect your daily quota: small batches, paused schedules, no unnecessary full refreshes.
- Cite the official documentation pages you used in your README.
- If a feature is unavailable in your workspace, tell your trainer the same day and document the alternative you used.

### Your assignment

| Field | To be completed with your trainer |
|---|---|
| Trainee number (NN) | |
| Business unit | |
| Source tables (samples catalog) | |
| Row-filter dimension | |
| Columns to mask | |
| Visual data prep question | |
| KPIs | |